

COURS COMPLET : ARCHITECTURES DE DEEP LEARNING POUR LES SÉRIES TEMPORELLES

De l'intuition aux mathématiques
LSTM, GRU, CNN/TCN et MLP expliqués pédagogiquement

Cours de Compréhension Approfondies

Décembre 2025

Table des matières

1	INTRODUCTION : POURQUOI CES MODÈLES ?	7
1.1	Le Problème : Prédire le Futur à Partir du Passé	7
1.2	Types de Modèles	7
2	COMPRENDRE LES RÉSEAUX RÉCURRENTS (RNN) BASIQUES	9
2.1	Idée Fondamentale : La Boucle Récurrente	9
2.1.1	Analogie Simple : La Conversation	9
2.1.2	Équation Mathématique Basique	10
2.2	Le Grand Problème : Vanishing Gradient	10
2.2.1	Qu'est-ce que c'est ?	10
2.2.2	Mathématiquement	10
3	SOLUTION 1 : LSTM (Long Short-Term Memory)	13
3.1	L'Idée Géniale : Deux Chemins Différents	13
3.2	Architecture Détaillée d'une Cellule LSTM	13
3.2.1	Étape 1 : Forget Gate (Porter d'Oubli)	14
3.2.2	Étape 2 : Input Gate (Porte d'Entrée) et Candidat	14
3.2.3	Étape 3 : Mise à Jour du Cell State	14
3.2.4	Étape 4 : Output Gate (Porte de Sortie)	15
3.3	Résumé des Équations LSTM	15
3.4	Pourquoi LSTM Capture les Dépendances Long-Terme	16
3.4.1	Propagation Forward	16
3.4.2	Backpropagation Through Time (BPTT)	16
3.5	Nombre de Paramètres du LSTM	17
4	SOLUTION 2 : GRU (Gated Recurrent Unit)	19
4.1	L'Idée : Simplifier le LSTM	19
4.1.1	Changements Clés	19
4.2	Architecture du GRU	19
4.2.1	Reset Gate	19
4.2.2	Update Gate	20

4.2.3	Candidat Hidden State	20
4.2.4	Mise à Jour de l'État Caché	20
4.3	Résumé et Comparaison GRU vs LSTM	20
5	SOLUTION 3 : CNN et TCN (Temporal Convolutional Networks)	23
5.1	Changement de Paradigme : Pas de Récurrence	23
5.2	Convolution 1D	23
5.2.1	Concept Basique	23
5.2.2	Mathématiquement	24
5.3	Le Problème : Réceptive Field Trop Petit	24
5.4	Solution Magique : Dilated Convolutions	24
5.4.1	L'Idée	24
5.4.2	Exemples Concrets	25
5.4.3	La Magie : RF Exponentielle	25
5.5	Causal Convolutions	25
5.5.1	Concept	26
5.5.2	Implémentation	26
5.6	Temporal Convolutional Networks (TCN)	26
5.6.1	Architecture Générale	26
5.6.2	Batch Normalization	27
5.6.3	Residual Connections (Skip Connections)	27
5.7	Avantages du TCN vs LSTM	27
5.8	Nombre de Paramètres du TCN	27
6	SOLUTION 4 : MLP (Multilayer Perceptron)	29
6.1	Le Modèle Simple : Réseau Dense	29
6.1.1	Pas de Structure Temporelle	29
6.2	Formatage des Données pour MLP	29
6.2.1	Fenêtre Glissante	29
6.2.2	Flatten (Aplatissement)	30
6.3	Architecture Basique	30
6.3.1	Équations	30
6.3.2	Exemple Concret	30
6.4	Limitation Majeure : Perte d'Information Temporelle	30
6.4.1	Problème	30
6.4.2	Solutions Possibles	31
6.5	Quand Utiliser un MLP ?	31
6.5.1	Cas d'Usage	31
6.5.2	Performance Relative	31

7	COMPARAISON COMPLÈTE : QUAND CHOISIR QUOI ?	33
7.1	Table Synthétique de Comparaison	33
7.2	Recommandation pour Données Financières	33
7.2.1	1er Choix : TCN	33
7.2.2	2e Choix : Ensemble LSTM + TCN	34
7.2.3	À Éviter	34
8	CONCEPTS AVANCÉS	35
8.1	Apprentissage sur Plusieurs Horizons	35
8.1.1	Problème	35
8.1.2	Recommandation	35
8.2	Walk-Forward Validation	35
8.3	Métriques d'Évaluation	36
8.3.1	RMSE (Root Mean Squared Error)	36
8.3.2	MAE (Mean Absolute Error)	36
8.3.3	MAPE (Mean Absolute Percentage Error)	36
8.3.4	MASE (Mean Absolute Scaled Error)	36
9	IMPLÉMENTATION PRATIQUE	37
9.1	Code TCN en TensorFlow/Keras	37
9.2	Code LSTM en TensorFlow/Keras	38
10	RESSOURCES POUR APPROFONDIR	39
10.1	Papiers Scientifiques Clés	39
10.2	Ressources Pédagogiques	40
10.3	Implémentations et Code	40
11	CONCLUSION ET PROCHAINES ÉTAPES	41
11.1	Résumé Conceptuel	41
11.2	Pour Votre Problème (Prédiction Prix 1-14 Jours)	41
11.3	Prochaines Étapes d'Apprentissage	42

Chapitre 1

INTRODUCTION : POURQUOI CES MODÈLES ?

1.1 Le Problème : Prédire le Futur à Partir du Passé

Imaginez que vous regardez le prix d'une action stock. Chaque jour, vous observez :

- Prix d'ouverture (Open)
- Prix maximal (High)
- Prix minimal (Low)
- Prix de clôture (Close)
- Volume échangé (Volume)

Question : Comment utiliser *tout l'historique passé* pour prédire le prix demain, dans 3 jours, ou dans 2 semaines ?

C'est un problème de **séries temporelles** (time series). Contrairement aux images où chaque pixel est indépendant, ici :

- L'ordre des données **importe beaucoup**
- Le passé lointain (il y a 20 jours) peut influencer l'avenir
- Le passé récent (hier) influence beaucoup plus directement l'avenir
- Les données ont une **structure temporelle**

1.2 Types de Modèles

Pour résoudre ce problème, il existe 4 architectures principales :

1. **LSTM** (Long Short-Term Memory) : Une RNN (Réseau Récurrent) avec mémoire sophistiquée
2. **GRU** (Gated Recurrent Unit) : Variante simplifiée du LSTM, plus rapide
3. **CNN/TCN** (Temporal Convolutional Networks) : Approche convolutionnelle avec "vue large"

4. **MLP** (Multilayer Perceptron) : Simple réseau dense

Chacun a ses forces et faiblesses. Notre objectif : **comprendre comment ils fonctionnent**.

Intuition Clé

Tous ces modèles répondent à la même question : « *Comment extraire l'information utile du passé pour prédire le futur ?* » Ils diffèrent seulement dans **comment** ils gèrent cette information.

Chapitre 2

COMPRENDRE LES RÉSEAUX RÉCURRENTS (RNN) BASIQUES

2.1 Idée Fondamentale : La Boucle Récurrente

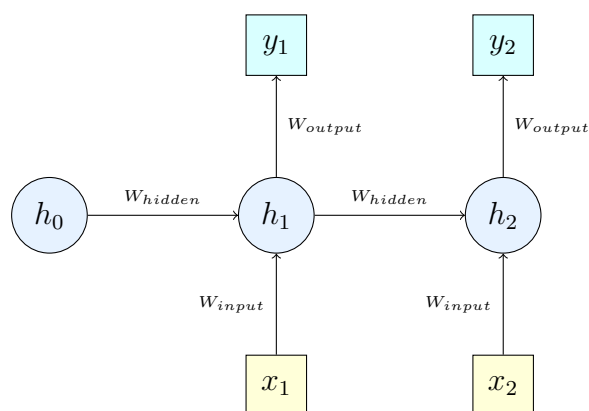
Contrairement aux réseaux standards (feed-forward), les réseaux récurrents ont une **boucle** qui permet à l'information de circuler d'un pas de temps à l'autre.

2.1.1 Analogie Simple : La Conversation

Imaginez une conversation entre deux personnes.

Réseau standard (feed-forward) : Chaque phrase est traitée indépendamment, sans mémoire de ce qui a été dit avant.

Réseau récurrent (RNN) : Chaque phrase est comprise *en fonction du contexte* — ce qui a été dit avant. Vous gardez une « mémoire court-terme » (hidden state) qui se met à jour à chaque phrase.



Chaque cellule RNN reçoit :

- L'**input** actuel : x_t (prix du jour t)
- L'**état** caché du pas précédent : h_{t-1} (mémoire)

Et produit :

- Un **nouvel état caché** : h_t (mémoire mise à jour)
- Une **prédiction** : y_t

2.1.2 Équation Mathématique Basique

$$h_t = \tanh(W_{input} \cdot x_t + W_{hidden} \cdot h_{t-1} + b) \quad (2.1)$$

où :

- $\tanh$: fonction d'activation (comprime entre -1 et 1)
- W_{input} , W_{hidden} : matrices de poids
- b : biais

Interprétation : L'état caché h_t est une **combinaison** de :

- Ce qui arrive maintenant (x_t)
- Ce qu'on savait avant (h_{t-1})

2.2 Le Grand Problème : Vanishing Gradient

2.2.1 Qu'est-ce que c'est ?

Lors de l'entraînement, on doit calculer les gradients (dérivées) pour mettre à jour les poids. Pour les RNN, il faut « remonter le temps » via la **backpropagation through time (BPTT)**.

Analogie : Le Téléphone Cassé

Imaginez 20 personnes en ligne qui se passent un message d'oreille à oreille (comme au jeu du téléphone). À chaque passage, le message s'affaiblit (multiplié par un petit nombre). Après 20 passages, le message est complètement inaudible.

C'est exactement ce qui se passe avec les gradients : après backprop sur 20 pas de temps, le gradient « d'hier » est tellement petit qu'il ne peut plus mettre à jour les poids.

2.2.2 Mathématiquement

Pour l'entraînement, on calcule :

$$\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(z) \cdot W_{hidden} \quad (2.2)$$

où $z = W_{input} \cdot x_t + W_{hidden} \cdot h_{t-1} + b$.

La dérivée de $\tanh'(z)$ est toujours entre 0 et 1. De plus, $\|W_{hidden}\|$ est souvent < 1 .

Résultat : Quand on chaîne ces multiplications sur 20 pas :

$$\frac{\partial h_{20}}{\partial h_0} = \prod_{t=1}^{20} \tanh'(z_t) \cdot W_{hidden} \quad (2.3)$$

Chaque terme est < 1 , donc le produit de 20 termes < 1 devient très petit, très proche de 0.

Conséquence : Les poids de la couche d'entrée (ancienne information) reçoivent des mises à jour microscopiques et n'apprennent pas.

Problème Critique

Avec les RNN standards, on ne peut pas apprendre des **dépendances de long terme**. L'information d'il y a 20 jours ne peut pas influencer la prédiction d'aujourd'hui, car le gradient s'est évanoui.

Chapitre 3

SOLUTION 1 : LSTM (Long Short-Term Memory)

3.1 L'Idée Géniale : Deux Chemins Différents

Pour éviter le vanishing gradient, les chercheurs (Hochreiter & Schmidhuber, 1997) ont eu une idée brillante :

Au lieu d'une seule boucle, utiliser deux boucles parallèles :

1. **Cell State** (c_t) : Une « mémoire à long terme » — l'information passe presque inchangée d'un pas au suivant
2. **Hidden State** (h_t) : Une « mémoire à court terme » — ce qui sort vraiment du réseau

Analogie : La Rivière et les Portes

Imaginez une rivière qui coule à travers le temps. Le cell state est comme l'eau qui coule dans la rivière — elle s'écoule presque librement, sans trop de friction.

Mais il y a des **portes** qui contrôlent quoi faire de cette eau :

- **Forget Gate** : « Dois-je oublier une partie de l'eau ? »
- **Input Gate** : « Dois-je ajouter de la nouvelle eau ? »
- **Output Gate** : « Combien d'eau dois-je laisser sortir ? »

3.2 Architecture Détaillée d'une Cellule LSTM

Une cellule LSTM traite un élément d'une séquence. Elle reçoit :

- Input courant : x_t
- État caché du pas précédent : h_{t-1}
- Cell state du pas précédent : c_{t-1}

Et produit :

- Nouvel état caché : h_t
- Nouveau cell state : c_t

3.2.1 Étape 1 : Forget Gate (Porter d'Oubli)

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (3.1)$$

où σ est la fonction **sigmoid** : $\sigma(z) = \frac{1}{1+e^{-z}}$.

Propriété de sigmoid : Elle compresse toute entrée entre 0 et 1.

Interprétation :

- Si $f_t \approx 1$: « Garde toute l'information du passé »
- Si $f_t \approx 0$: « Oublie complètement le passé »
- En pratique : $f_t \approx 0.7$ signifie « Garde 70% du passé, oublie 30% »

Exemple concret :

- Prix d'action stable $\Rightarrow$ forget gate ≈ 1 (garde le trend)
- Krach boursier $\Rightarrow$ forget gate ≈ 0 (oublie l'ancien régime)

3.2.2 Étape 2 : Input Gate (Porte d'Entrée) et Candidat

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (3.2)$$

$$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad (3.3)$$

Interprétation :

- i_t : « Quelle fraction de la nouvelle information vais-je intégrer ? »
- $\tilde{c}_t$: « Quelle est cette nouvelle information ? » (candidat pour le cell state)

3.2.3 Étape 3 : Mise à Jour du Cell State

C'est l'équation cruciale — elle **résout le vanishing gradient** :

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (3.4)$$

où $\odot$ signifie « multiplication élément par élément » (produit de Hadamard).

Pourquoi c'est magique :

Au lieu d'une multiplication de matrices :

$$c_t = M \cdot c_{t-1}$$

on a une **addition** :

$$c_t = f_t \cdot c_{t-1} + i_t \cdot \tilde{c}_t$$

Quand on calcule les gradients en arrière :

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t \quad (\text{au lieu d'une matrice}) \quad (3.5)$$

Le miracle : f_t est souvent proche de 1 (car le réseau apprend à maintenir les informations importantes). Donc le gradient **ne s'effondre pas** !

Pourquoi Cela Fonctionne

Dans un RNN standard, les gradients sont multipliés par des matrices pleines. Après 20 multiplications, ils s'effondrent.

Dans un LSTM, les gradients sont multipliés par des vecteurs f_t dont les composants sont proches de 0 ou 1. Le réseau **apprend** quels composants doivent être ≈ 1 (pour préserver l'info) et lesquels doivent être ≈ 0 (pour oublier).

C'est comme si le réseau avait ses propres « vannes » pour contrôler le flux du gradient.

3.2.4 Étape 4 : Output Gate (Porte de Sortie)

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (3.6)$$

$$h_t = o_t \odot \tanh(c_t) \quad (3.7)$$

Interprétation :

- o_t : « Quelle partie du cell state (mémoire long-terme) dois-je exposer au dehors ? »
- h_t : L'état caché « court-terme » envoyé au pas suivant et à la couche de prédiction

3.3 Résumé des Équations LSTM

Pour récapituler, une cellule LSTM exécute cet ensemble d'équations :

Les 4 Étapes d'une Cellule LSTM

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad (\text{Forget}) \quad (3.8)$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (\text{Input}) \quad (3.9)$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c) \quad (\text{Candidat}) \quad (3.10)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (\text{Cell State}) \quad (3.11)$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad (\text{Output}) \quad (3.12)$$

$$h_t = o_t \odot \tanh(c_t) \quad (\text{Hidden State}) \quad (3.13)$$

3.4 Pourquoi LSTM Capture les Dépendances Long-Terme

3.4.1 Propagation Forward

Grâce à l'addition au lieu de la multiplication, l'information du passé lointain peut traverser intact :

$$c_T = f_T \odot f_{T-1} \odot \cdots \odot f_1 \odot c_0 + (\text{contributions récentes}) \quad (3.14)$$

Si les forget gates apprennent à rester ≈ 1 , alors le produit $f_T \odot f_{T-1} \odot \cdots$ reste ≈ 1 au lieu de s'effondrer.

3.4.2 Backpropagation Through Time (BPTT)

Même chose en arrière : le gradient du cell state circule facilement :

$$\frac{\partial \mathcal{L}}{\partial c_t} = \frac{\partial \mathcal{L}}{\partial c_{t+1}} \cdot f_{t+1} + \frac{\partial \mathcal{L}}{\partial h_t} \quad (3.15)$$

Le terme f_{t+1} reste grand (proche de 1) si le réseau le décide, donc le gradient ne disparaît pas.

Capacité du LSTM

Un LSTM peut mémoriser les événements importants pendant 100+ pas de temps, tandis qu'un RNN standard s'effondre après 5-10 pas.

C'est comme la différence entre une personne ayant la mémoire d'un poisson (RNN) et une personne ayant une excellente mémoire à long terme (LSTM).

3.5 Nombre de Paramètres du LSTM

Pour une couche LSTM avec :

- Taille d'entrée : d_{in} (5 pour OHLCV)
- Taille d'état caché : d_h (ex : 64)

Chaque porte (forget, input, output) + candidat = 4 « copies » de poids :

$$\text{Params}_{LSTM} = 4 \times (d_{in} \cdot d_h + d_h \cdot d_h + d_h) = 4d_h(d_{in} + d_h + 1) \quad (3.16)$$

Exemple concret :

- $d_{in} = 5, d_h = 64 \Rightarrow \text{Params} = 4 \times 64 \times 70 = 17,920$ (1 couche)
- 2 couches LSTM : $\approx 50,000$ paramètres

Chapitre 4

SOLUTION 2 : GRU (Gated Recurrent Unit)

4.1 L’Idée : Simplifier le LSTM

En 2014, Cho et al. ont pensé : « Le LSTM fonctionne bien, mais c’est compliqué. Peut-on simplifier ? »

Résultat : Le **GRU** combine certaines portes du LSTM en gardant seulement les essentiels.

4.1.1 Changements Clés

- **Un seul état caché** : h_t (au lieu de deux : h_t et c_t)
- **Deux portes au lieu de trois** : reset gate et update gate
- **Paramètres réduits** : 25% moins de poids (3/4 du LSTM)
- **Plus rapide à entraîner** : 30% plus rapide empiriquement

Analogie

Si LSTM est une voiture de luxe avec 4 portes and un coffre séparé, GRU est une voiture compacte avec 2 portes. Elle fait le même travail (te transporter), juste un peu moins sophistiquée.

4.2 Architecture du GRU

4.2.1 Reset Gate

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r) \quad (4.1)$$

Sens : « Dois-je oublier complètement l'état passé ? » (0 = oublier, 1 = garder)

Différence avec LSTM : C'est une version du « forget gate » du LSTM.

4.2.2 Update Gate

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z) \quad (4.2)$$

Sens : « Quelle fraction du nouvel état candidat dois-je utiliser ? »

4.2.3 Candidat Hidden State

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h) \quad (4.3)$$

Clé : Le reset gate r_t **filtre** d'abord l'ancien état avant de calculer le candidat.

4.2.4 Mise à Jour de l'État Caché

$$h_t = (1 - z_t) \odot \tilde{h}_t + z_t \odot h_{t-1} \quad (4.4)$$

Interprétation : Interpolation entre :

- Nouvel état candidat : $\tilde{h}_t$
- Ancien état : h_{t-1}

Le poids de cette interpolation est z_t (update gate).

Si $z_t \approx 1$: Garde pratiquement l'ancien état (change lentement) **Si** $z_t \approx 0$: Adopte le nouvel état candidat (change rapidement)

4.3 Résumé et Comparaison GRU vs LSTM

Caractéristique	LSTM	GRU
Nombre de portes	3 (f, i, o)	2 (r, z)
Nombre d'états	2 (h_t , c_t)	1 (h_t)
Paramètres (1 couche)	$4d_h(d_{in} + d_h + 1)$	$3d_h(d_{in} + d_h + 1)$
Params relatif	100%	75%
Vitesse (empirique)	Baseline	+30% plus rapide
Performance	Excellente	Généralement similaire
Résout vanishing gradient	✓	✓

Quand Utiliser Quoi ?

- **LSTM** : Quand vous avez beaucoup de GPU, données massives, besoin de performance maximale
- **GRU** : Quand vous voulez entraîner vite, déployer sur edge devices, ou simple compétition de rapidité

Empiriquement, les deux performant très similairement sur les données financières. GRU est souvent meilleur pour le rapport performance/vitesse.

Chapitre 5

SOLUTION 3 : CNN et TCN (Temporal Convolutional Networks)

5.1 Changement de Paradigme : Pas de Récurrence

LSTM et GRU sont des **réseaux récurrents** : à chaque pas de temps, ils traitent un élément à la fois en ordre.

Nouvelle idée (Bai et al., 2018) : Pourquoi ne pas traiter **toute la séquence en parallèle** comme les images ?

Analogie : Regarder vs Analyser

Imaginez une photo d'une route avec 20 voitures.

RNN : Vous regardez chaque voiture une par une, de gauche à droite, en gardant une mémoire mentale.

CNN : Vous regardez *toute la photo à la fois*, en cherchant des patterns locaux (ex : les voitures rouges, les intersections, etc.)

5.2 Convolution 1D

Les images sont 2D (hauteur $\times$ largeur). Pour les séries temporelles, on utilise des convolutions **1D** (seulement du temps).

5.2.1 Concept Basique

Imaginez un filtre de taille 3 qui « glisse » sur la séquence :

Séquence : $[x_1, x_2, x_3, x_4, x_5, \dots]$

Filtre : $[w_1, w_2, w_3]$

Positions :

- Position 1 : $w_1x_1 + w_2x_2 + w_3x_3$
- Position 2 : $w_1x_2 + w_2x_3 + w_3x_4$
- Position 3 : $w_1x_3 + w_2x_4 + w_3x_5$

Chaque position produit un nombre. La sortie est plus courte que l'entrée.

Réceptive field (RF) : Nombre d'éléments d'entrée qu'une position de sortie « voit ». Ici : 3.

5.2.2 Mathématiquement

Pour une convolution 1D à position t :

$$y_t = \sum_{k=0}^{K-1} w_k \cdot x_{t+k} + b \quad (5.1)$$

où K est la taille du filtre (kernel size).

5.3 Le Problème : Réceptive Field Trop Petit

Si on empile plusieurs convolutions 1D :

- Couche 1 (kernel=3) : RF = 3
- Couche 2 (kernel=3) : RF = 3 + 3 - 1 = 5
- Couche 3 (kernel=3) : RF = 5 + 3 - 1 = 7

Problème : Pour avoir RF = 30 (voir 30 pas de temps), il faudrait 15+ couches. C'est profond et lent.

Limitation des Conv Basiques

Une convolution 1D standard augmente son RF **linéairement** avec la profondeur.
Pour voir 30 pas de temps, il faut 30+ couches. C'est inefficace.

5.4 Solution Magique : Dilated Convolutions

5.4.1 L'Idée

Au lieu de multiplier les poids par des éléments **consécutifs**, on les multiplie par des éléments **espacés**.

$$y_t = \sum_{k=0}^{K-1} w_k \cdot x_{t+d \cdot k} + b \quad (5.2)$$

où d est le **dilation factor**.

5.4.2 Exemples Concrets

Dilation = 1 (convolution normale) :

$$y_t = w_0x_t + w_1x_{t+1} + w_2x_{t+2}$$

RF = 3

Dilation = 2 :

$$y_t = w_0x_t + w_1x_{t+2} + w_2x_{t+4}$$

RF = 1 + 2(3 - 1) = 5

Dilation = 4 :

$$y_t = w_0x_t + w_1x_{t+4} + w_2x_{t+8}$$

RF = 1 + 4(3 - 1) = 9

5.4.3 La Magie : RF Exponentielle

Si on empile :

- Couche 1 : dilation = 1, RF = 3
- Couche 2 : dilation = 2, RF = 3 + 2 × (3 - 1) = 7
- Couche 3 : dilation = 4, RF = 3 + 4 × (3 - 1) = 11
- Couche 4 : dilation = 8, RF = 3 + 8 × (3 - 1) = 19

Avec seulement 4 couches et dilation en progression {1, 2, 4, 8}, on a RF = 19!

Formule générale :

$$\text{RF} = 1 + 2 \sum_{l=0}^{L-1} (K - 1) \times 2^l = 1 + 2(K - 1)(2^L - 1) \quad (5.3)$$

Croissance : Avec $K = 3$, $L = 5$ (5 couches) $\Rightarrow$ RF = 1 + 4 × 31 = 125!

Pourquoi C'est Révolutionnaire

Les dilated convolutions augmentent le RF **exponentiellement** au lieu de linéairement. - Convolutions normales : 30 couches pour RF=30 - Dilated convolutions : 4-5 couches pour RF=30+

C'est une révolution pour les séries temporelles longues!

5.5 Causal Convolutions

Pour faire des prédictions, il ne faut jamais utiliser d'information **future**. On doit donc garder seulement le passé.

5.5.1 Concept

Au lieu de :

$$y_t = w_0x_{t-1} + w_1x_t + w_2x_{t+1}$$

(qui utilise x_{t+1} : l'avenir !)

On utilise :

$$y_t = w_0x_{t-2} + w_1x_{t-1} + w_2x_t$$

(seulement le passé)

Mathématiquement, on dit que les positions d'entrée sont « causales » (causales = ne dépend que du passé).

5.5.2 Implémentation

On « décale » les entrées avant la convolution pour s'assurer qu'on ne voit jamais le futur.

5.6 Temporal Convolutional Networks (TCN)

Un TCN combine :

1. Convolutions dilatées
2. Causal padding
3. Residual connections
4. Batch normalization

5.6.1 Architecture Générale

1. **Bloc résiduel 1** : Conv dilatée (d=1) + BN + ReLU + Conv (d=1) + BN + Residual
2. **Bloc résiduel 2** : Conv dilatée (d=2) + BN + ReLU + Conv (d=2) + BN + Residual
3. **Bloc résiduel 3** : Conv dilatée (d=4) + BN + ReLU + Conv (d=4) + BN + Residual
4. **Global Average Pooling** : Moyenne sur le temps
5. **Dense output** : Une prédiction

5.6.2 Batch Normalization

Entre les couches, on applique une normalisation :

$$\text{BN}(x) = \gamma \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta \quad (5.4)$$

Effet : Stabilise l'entraînement, permet des learning rates plus élevés, réduit l'internal covariate shift.

5.6.3 Residual Connections (Skip Connections)

Au lieu de :

$$h_{l+1} = f(h_l)$$

On utilise :

$$h_{l+1} = f(h_l) + h_l$$

Avantage : Permet aux gradients de « sauter » les couches, facilitant le flux du gradient.

5.7 Avantages du TCN vs LSTM

Propriété	LSTM	TCN
Parallélisable	Non (séquentiel)	Oui
Traitement	Pas à pas	Tout d'un coup
RF maximal	Illimité théoriquement	Illimité avec dilation
Paramètres (même RF)	Nombreux	Moins
Latence inference	Élevée (2-3ms/sample)	Basse (0.5-1ms)
Mémoire train	Élevée	Modérée
Inductive bias	Mémoire explicite	Patterns locaux

Quand Utiliser TCN vs LSTM ?

- **TCN** : Séries temporelles de longueur modérée (20-100 pas), besoin de latence basse, dépendances "locales" importantes
- **LSTM** : Très longues séquences (1000+ pas), mémoire explicite crucial, ressources computationnel élevées disponibles

Pour le trading (1-14 jours), TCN est généralement supérieur.

5.8 Nombre de Paramètres du TCN

Pour chaque bloc avec convolution (dilation, kernel=3, filters=F) :

$$\text{Params}_{\text{block}} \approx 2 \times (3 \times F^2 + F) + \text{BN} \quad (5.5)$$

Avec 3 blocs et $F=64$:

$$\text{Params}_{\text{TCN}} \approx 3 \times 2 \times (3 \times 64^2 + 64) \approx 49,000 \quad (5.6)$$

Comparaison :

- TCN 3 blocs : 49,000 params
- LSTM 2 couches : 51,000 params
- Similaire, mais TCN parallélisable !

Chapitre 6

SOLUTION 4 : MLP (Multilayer Perceptron)

6.1 Le Modèle Simple : Réseau Dense

Un MLP est simplement un réseau de neurones « classique » : des couches denses (fully connected).

6.1.1 Pas de Structure Temporelle

Contrairement à LSTM/GRU/CNN, un MLP **ne « comprend » pas l'ordre temporel**.

Exemple : Si vous donnez la séquence $[x_1, x_2, x_3]$ en la « flattenissant » à $[x_1, x_2, x_3]$ et aussi sa version permutée $[x_2, x_1, x_3]$, un MLP les traiterait de manière identique (après réorganisation des poids).

6.2 Formatage des Données pour MLP

6.2.1 Fenêtre Glissante

Pour 20 pas de temps et 5 features :

Entrée (shape = 20×5) :

$$\begin{pmatrix} \text{Open}_1 & \text{High}_1 & \text{Low}_1 & \text{Close}_1 & \text{Vol}_1 \\ \text{Open}_2 & \text{High}_2 & \dots & & \\ \vdots & & & & \\ \text{Open}_{20} & & & \text{Close}_{20} & \text{Vol}_{20} \end{pmatrix} \quad (6.1)$$

6.2.2 Flatten (Aplatissement)

On transforme la matrice 20×5 en vecteur de 100 éléments :

$$[\text{Open}_1, \text{High}_1, \dots, \text{Vol}_1, \text{Open}_2, \dots, \text{Vol}_{20}] \quad (6.2)$$

Input layer : 100 neurones

6.3 Architecture Basique

6.3.1 Équations

Pour chaque couche l :

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)} \quad (6.3)$$

$$a^{(l)} = \text{activation}(z^{(l)}) \quad (6.4)$$

Activations courantes :

- **ReLU** : $f(z) = \max(0, z)$
- **Tanh** : $f(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$
- **Sigmoid** : $f(z) = \frac{1}{1 + e^{-z}}$

6.3.2 Exemple Concret

Pour prédiction avec 2 couches cachées :

- Input : 100 (20 temps $\times$ 5 features)
- Couche 1 : 64 neurones, activation ReLU
- Couche 2 : 32 neurones, activation ReLU
- Output : 1 neurone, activation linéaire (pour régression)

Nombre de paramètres :

$$(100 + 1) \times 64 + (64 + 1) \times 32 + (32 + 1) \times 1 = 6,464 + 2,080 + 33 = 8,577 \quad (6.5)$$

6.4 Limitation Majeure : Perte d'Information Temporelle

6.4.1 Problème

L'ordre temporel est implicite dans la position des éléments du vecteur. Mais un MLP ne « comprend » pas naturellement cette structure.

Exemple concret :

- Séquence réelle : Prix stable $\rightarrow$ Puis krach $\rightarrow$ Rebond
- Séquence permutée : Krach $\rightarrow$ Rebond $\rightarrow$ Stable

Un MLP standard traiterai les deux différemment **si on lui donne les positions**, mais pas si le réseau ne le décide pas.

6.4.2 Solutions Possibles

1. **Positional Encoding** : Ajouter l'index de temps comme feature supplémentaire
2. **Time Lags Explicites** : Ajouter des features à décalages temporels
3. **Attention** : Laisser le réseau « choisir » quelles positions regarder
4. **En tant que « head »** : Utiliser MLP après LSTM/CNN pour les prédictions finales

6.5 Quand Utiliser un MLP ?

6.5.1 Cas d'Usage

- **Baseline** : Pour vérifier si un problème est résolvable
- **Head prédictif** : Après LSTM/CNN pour transformer latent features en prédictions
- **Features engineered** : Avec des features pré-calculées (moyennes mobiles, ratios, etc.)
- **Données non-séquentielles** : Après avoir extrait des features fixes de la séquence

6.5.2 Performance Relative

- **Horizon court-terme (1-2 jours)** : MLP peut être compétitif si bien engineered
- **Horizon moyen (3-7 jours)** : LSTM/GRU/TCN supérieurs
- **Horizon long-terme (14+ jours)** : TCN > LSTM/GRU $\gg$ MLP

Chapitre 7

COMPARAISON COMPLÈTE : QUAND CHOISIR QUOI ?

7.1 Table Synthétique de Comparaison

Critère	LSTM	GRU	TCN	MLP
Dépendances long-terme	✓✓	✓✓	✓✓	×
Parallélisabilité	×	×	✓✓	✓✓
Paramètres	Nombreux	Moyen	Moyen	Peu
Latence inference	Élevée	Élevée	Basse	Très basse
Mémoire training	Très élevée	Élevée	Modérée	Basse
Temps training	Lent	Moyen	Rapide	Très rapide
Robustesse bruit	✓	✓	✓✓	×
Overfitting risk	Élevé	Élevé	Faible	Très faible
Implémentation	Complexe	Complexe	Modérée	Simple

7.2 Recommandation pour Données Financières

Pour prédire les prix d'actions/crypto sur 1-14 jours :

7.2.1 1er Choix : TCN

Raisons :

- RF exponentielle adaptée à 20 pas
- Parallélisable = déploiement rapide
- Latence basse = trading real-time possible
- Robustesse bruit financier excellente

7.2.2 2e Choix : Ensemble LSTM + TCN

Raisons :

- TCN pour performance/latence
- LSTM pour robustesse/mémoire
- Moyenne pondérée = stabilité maximale

Poids recommandés : 60% TCN + 40% LSTM

7.2.3 À Éviter

- **GRU seul** : Plus lent que TCN, pas meilleur que LSTM
- **MLP seul** : Perd information temporelle
- **RNN classique** : Vanishing gradient = catastrophe

Chapitre 8

CONCEPTS AVANCÉS

8.1 Apprentissage sur Plusieurs Horizons

8.1.1 Problème

On veut prédire : 1 jour, 3 jours, 5 jours, 7 jours, 14 jours à l'avance.

Stratégies :

1. **Direct** : H modèles séparés (un par horizon)
2. **Recursive** : 1 modèle, appliqué H fois
3. **MIMO** : 1 modèle, sortie H -dimensionnelle
4. **Hybrid** : TCN pour horizons clés (1, 3, 7, 14), interpolation pour autres

8.1.2 Recommandation

Approche **Hybrid Direct** :

- Entraîner TCN direct pour horizons 1, 3, 7, 14
- Pour d'autres (2, 5, 6, ...), interpoler linéairement
- Ensemble LSTM/TCN pour chaque horizon

8.2 Walk-Forward Validation

Pour évaluer sans data leakage :

1. Split 1 : Train $[0, t_0]$, Test $[t_0, t_0 + w]$
2. Split 2 : Train $[0, t_0 + w]$, Test $[t_0 + w, t_0 + 2w]$
3. Continuer jusqu'à fin données

Avantage : Simule vraiment le déploiement (apprendre du passé, prédire le futur).

8.3 Métriques d'Évaluation

8.3.1 RMSE (Root Mean Squared Error)

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (8.1)$$

Sensible aux outliers (gros erreurs pénalisées beaucoup).

8.3.2 MAE (Mean Absolute Error)

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (8.2)$$

Robuste au bruit. **Recommandé pour finance.**

8.3.3 MAPE (Mean Absolute Percentage Error)

$$\text{MAPE} = \frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \% \quad (8.3)$$

Normalise par la magnitude. Bon pour comparer différentes séries.

8.3.4 MASE (Mean Absolute Scaled Error)

$$\text{MASE} = \frac{\text{MAE}}{\text{MAE}_{naive}} \quad (8.4)$$

Compares à une baseline naïve. **Meilleure pour comparaisons rigoureuses.**

Chapitre 9

IMPLÉMENTATION PRATIQUE

9.1 Code TCN en TensorFlow/Keras

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

def build_tcn(input_shape, output_size=1):
    inputs = keras.Input(shape=input_shape)

    # Bloc TCN 1 (dilation=1)
    x = layers.Conv1D(64, kernel_size=3,
                      padding='causal', dilation_rate=1)(inputs)
    x = layers.BatchNormalization()(x)
    x = layers.ReLU()(x)
    x = layers.Dropout(0.2)(x)
    x = layers.Conv1D(64, kernel_size=3,
                      padding='causal', dilation_rate=1)(x)
    x = layers.BatchNormalization()(x)
    residual = layers.Conv1D(64, kernel_size=1)(inputs)
    x = layers.Add()([x, residual])
    x = layers.ReLU()(x)

    # Bloc TCN 2 (dilation=2)
    # ... (similaire avec dilation=2)

    # Bloc TCN 3 (dilation=4)
    # ... (similaire avec dilation=4)

    # Output
```

```
x = layers.GlobalAveragePooling1D()(x)
outputs = layers.Dense(output_size)(x)

model = keras.Model(inputs=inputs, outputs=outputs)
return model

# Utilisation
model = build_tcn(input_shape=(20, 5), output_size=1)
model.compile(optimizer='adam', loss='mae')
model.fit(X_train, y_train, validation_data=(X_val, y_val),
        epochs=100)
```

9.2 Code LSTM en TensorFlow/Keras

```
def build_lstm(input_shape, output_size=1):
    inputs = keras.Input(shape=input_shape)

    x = layers.LSTM(64, return_sequences=True)(inputs)
    x = layers.Dropout(0.3)(x)
    x = layers.LSTM(64, return_sequences=False)(x)
    x = layers.Dropout(0.3)(x)

    x = layers.Dense(32, activation='relu')(x)
    outputs = layers.Dense(output_size)(x)

    model = keras.Model(inputs=inputs, outputs=outputs)
    return model
```

Chapitre 10

RESSOURCES POUR APPROFONDIR

10.1 Papiers Scientifiques Clés

1. **LSTM Original** (1997)
 - Hochreiter, S., & Schmidhuber, J. (1997). "Long Short-Term Memory". *Neural Computation*, 9(8), 1735-1780.
 - URL : <https://direct.mit.edu/neco/article-abstract/9/8/1735>
 - **À lire** : Section 3 pour architecture LSTM détaillée
2. **GRU** (2014)
 - Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). "Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling". *arXiv :1412.3555*
 - URL : <https://arxiv.org/pdf/1412.3555.pdf>
 - **À lire** : Comparaison empirique LSTM vs GRU
3. **TCN** (2018)
 - Bai, S., Kolter, J. Z., & Koltun, V. (2018). "An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling". *arXiv :1803.01271*
 - URL : <https://arxiv.org/abs/1803.01271>
 - **À lire** : Comparaison TCN vs LSTM, dilated convolutions
4. **Vanishing Gradient** (2013)
 - Pascanu, R., Mikolov, T., & Bengio, Y. (2013). "On the Difficulty of Training Recurrent Neural Networks". *arXiv :1211.1541*
 - URL : <https://arxiv.org/abs/1211.1541>
 - **À lire** : Explication mathématique du vanishing gradient
5. **Batch Normalization** (2015)
 - Ioffe, S., & Szegedy, C. (2015). "Batch Normalization : Accelerating Deep Network Training by Reducing Internal Covariate Shift". *ICML*

- URL : <https://arxiv.org/abs/1502.03167>
- **À lire** : Pourquoi BN aide

10.2 Ressources Pédagogiques

1. Deep Learning Book

- Goodfellow, I., Bengio, Y., & Courville, A. (2016). "Deep Learning". MIT Press.
- Disponible gratuitement : <https://www.deeplearningbook.org/>
- **Chapitres** : 10 (RNN), 11 (LSTM/GRU)

2. Dive into Deep Learning (d2l.ai)

- Interactive course : <https://d2l.ai/>
- Excellents tutoriels LSTM, GRU, RNN

3. Vidéos YouTube

- StatQuest LSTM : <https://www.youtube.com/watch?v=YCzL96nL7j0>
- 3Blue1Brown Neural Networks : <https://www.youtube.com/watch?v=aircAruvnKk>

4. Colah's Blog

- "Understanding LSTM Networks" : <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>
- MEILLEURE introduction intuitive aux LSTM

10.3 Implémentations et Code

1. TensorFlow / Keras Documentation

- Time Series Forecasting : https://www.tensorflow.org/tutorials/structured_data/time_series

2. PyTorch Documentation

- LSTM Tutorial : https://pytorch.org/tutorials/beginner/nlp/sequence_models_tutorial.html

3. Darts (Deep Learning Time Series Library)

- TCN, NBeats, Transformers pré-implémentés
- <https://unit8.com/open-source/darts>

Chapitre 11

CONCLUSION ET PROCHAINES ÉTAPES

11.1 Résumé Conceptuel

Les 4 Architectures Résumées

- **LSTM** : Deux boucles (cell state + hidden state), portes d'oubli/entrée/sortie. Résout vanishing gradient. Excellent pour dépendances très long-terme.
- **GRU** : Simplification du LSTM. Une boucle, 2 portes. 75% des paramètres, performance similaire. Recommandé pour vitesse.
- **TCN** : Convolutions dilatées en cascade. RF exponentielle. Parallélisable, latence basse. Recommandé pour séries courtes-moyennes et trading.
- **MLP** : Réseau dense classique. Sans structure temporelle intrinsèque. Baseline ou head prédictif après LSTM/CNN.

11.2 Pour Votre Problème (Prédiction Prix 1-14 Jours)

Approche Recommandée :

1. **Modèle Principal** : TCN avec 3-4 blocs (dilations 1,2,4,8)
2. **Fallback** : LSTM 2 couches pour robustesse
3. **Ensemble** : 60% TCN + 40% LSTM
4. **Multi-horizon** : Modèles Direct pour 1, 3, 7, 14 jours
5. **Validation** : Walk-forward avec MAE/MASE

Résultats Attendus :

- Horizon 1 jour : MAE $\approx$ 2-3% du prix

- Horizon 14 jours : MAE $\approx$ 8-12% du prix

11.3 Prochaines Étapes d'Apprentissage

Si vous voulez approfondir :

1. **Comprenez le vanishing gradient**
 - Lisez : Pascanu et al. 2013
 - Essayez : Implémentez un RNN vanilla, observez le gradient disparaître
2. **Implémentez un LSTM from scratch**
 - Écrivez les 4 équations LSTM en NumPy
 - Puis en TensorFlow
3. **Comparez les architectures empiriquement**
 - Téléchargez vrai data (yfinance)
 - Entraînez LSTM, GRU, TCN sur le même problème
 - Mesurez latence, accuracy, temps training
4. **Explorez les architectures modernes**
 - Transformers (attention mechanism)
 - N-BEATS (architecture moderne spécialisée séries temporelles)

Dernier Conseil

La meilleure façon d'apprendre est **en faisant**. Implémentez, déboguez, expérimentez. Les équations mathématiques deviennent claires une fois qu'on les code.

Bon apprentissage !